Plan de Backend: Gorilla Grading

Documento de planificación · Fecha: 19 de octubre de 2026 · Repositorio: Gorilla_Grading_Prototypes_From_Scratch

Este documento recoge el estudio completo de la aplicación web actual y el plan para construir su backend. Se basa en la revisión del código (rutas, contextos, datos mock, formularios y flujos), la documentación de arquitectura y el roadmap del proyecto. No se ha modificado ningún archivo del código fuente.

1. Estado actual de la aplicación

La aplicación es una SPA de React y Vite sin backend. Todo lo que parece funcional está simulado con datos mock en `src/data/`.

Área	Hoy	Qué falta
Login	Cualquier código de 6 dígitos entra. La sesión es solo el email guardado en `localStorage` (LoginPage.tsx, AuthContext.tsx).	OTP real con códigos hasheados y sesión en cookie httpOnly.
Envío (wizard)	Se genera un ID aleatorio en el navegador. No se guarda nada.	Persistencia, pago y fotos de cartas.
Datos de envío (paso 3)	Los 5 campos no tienen estado ni atributo `name`, así que se pierden (SubmissionWizardPage.tsx).	Formulario conectado y validado.
Precio	Se calcula en el cliente.	Motor de precios en servidor.
Seguimiento	Busca en MOCK_ORDERS.	API e historial de custodia.
Verificación	Busca en MOCK_GRADED_CARDS.	Registro de certificados firmados.
Contacto y newsletter	Simulan el envío (ContactPage.tsx, NewsletterSection.tsx). No guardan nada.	Tablas y emails.
Mapa de equipo	Lee y escribe votos en api.restful-api.dev, un servicio público de terceros, sin autenticación (TeamVisualMapPage.tsx).	Retirarlo de producción o protegerlo con autenticación de staff.

Hallazgos que condicionan el diseño

1. **Certificado falso por defecto.** CertificateDetailPage.tsx (línea 31) muestra el certificado de Cade Cunningham para cualquier ID que no exista. En un producto cuya función es verificar, es el riesgo principal. Debe devolver 404.
2. **Los hashes no son reales.** El `securityHash` de los mocks (por ejemplo `e3b0c442…`) es el SHA-256 de una cadena vacía. No existe una firma verificable.
3. **Fuga potencial de datos personales.** El seguimiento contiene la dirección completa del cliente. Si la API pública devuelve eso por ID, expone datos personales. La respuesta pública no debe incluirla.
4. **La regla de nota no está definida, y los datos la contradicen.**
   - Cade Cunningham tiene nota 9.5 con la etiqueta "Gem Mint 10".

    - Black Lotus tiene una media de subgrados de 8.75 y nota 8.5.
    - Shanks tiene nota 10 con edges de 9.5.
5. **Reglas de negocio que no cuadran.**
    - El pedido GG-ORD-88219 tiene 3 cartas Express a 50 €, es decir 150 €, pero el total registrado es 126 €.
    - Una carta declarada en 8.500 € supera el límite Express de 3.000 € (mockServices.ts).
    - El wizard no valida `maxDeclaredValueEur`.
6. **Precio contradictorio con la web pública.** La página de precios indica "IVA 21 % y seguro incluidos" (LuxuryPricingSection.tsx, línea 693). El wizard suma seguro (máximo entre 8 € y el 0,8 % del valor declarado) y envío de 14,50 €, y no aparece IVA en el cálculo.
7. **Precios duplicados** en tres sitios: mockServices.ts, LuxuryPricingSection.tsx y ServicesPage.tsx.
8. **Configuración de despliegue.** vercel.json reescribe todas las rutas a index.html. Al añadir `/api`, hay que excluirla de esa regla.

2. Alcance del backend

El alcance del MVP corresponde a la Fase 1 del roadmap del proyecto:

- Cuentas de cliente con código de acceso por email (OTP).
- Pedidos reales con pago, direcciones y fotos de cartas.
- Seguimiento con historial de custodia, sin datos personales en la vista pública.
- Certificados emitidos, firmados y verificables por número, QR o NFC.
- Flujo de back-office: recepción, pre-inspección, grading, encapsulado y control de calidad con doble firma.
- Contacto, newsletter y eventos.

Quedan fuera del MVP la colección digital (Fase 2), la visión artificial e IA (Fase 3) y la API B2B (Fase 4). El modelo de datos debe permitirlas sin rehacerlo.

3. Arquitectura recomendada

Decisión	Recomendación	Motivo y alternativa
Lenguaje y API	TypeScript con Hono, desplegado en Vercel Functions, en una carpeta `api/` dentro del mismo repositorio.	Mismo lenguaje que el frontend y despliegue ya existente. Si la visión artificial pesa más adelante, puede ir como servicio aparte en Python.
Base de datos	PostgreSQL vía Vercel Marketplace (Neon) o Supabase.	Relacional, necesario para el historial y la integridad de los datos.
ORM y migraciones	Drizzle, con migraciones SQL versionadas.	Esquema tipado y migraciones revisables.
Validación	Zod, con schemas compartidos con el frontend.	Una sola fuente de verdad para los contratos.

Autenticación	OTP propio por email, con códigos hasheados, caducidad y límite de intentos. Sesión en cookie httpOnly.	Coincide con la interfaz actual. Alternativa: Clerk o Supabase Auth.
Pagos	Stripe Checkout.	La aplicación no toca datos de tarjeta (PCI SAQ A).
Email	Resend, con plantillas y tabla de outbox.	Reintentos y trazabilidad de cada envío.
Ficheros	Almacenamiento privado (Vercel Blob o Cloudflare R2) con URLs firmadas.	Las fotos de las cartas no deben ser públicas.
Tareas asíncronas	Tabla de jobs con cron de Vercel para reintentos.	Vercel Queues cuando haga falta; hoy está en beta.
Observabilidad	Logs estructurados y Sentry.	Diagnóstico de incidencias en producción.

Estructura propuesta: `api/` (endpoints finos), `server/` (dominio: precios, pedidos, grading, certificados, seguimiento), `db/` (esquema y migraciones) y `shared/` (schemas Zod y tipos).

4. Modelo de datos (PostgreSQL)

Tabla	Propósito	Notas clave
users	Cuentas	Email único (citext), idioma y rol: customer, staff, grader, qa, admin.
otp_challenges	Códigos de acceso	Solo se guarda el hash. Caducidad de 10 minutos y máximo 5 intentos.
sessions	Sesiones	Hash del token, expiración y revocación.
addresses	Direcciones	Vinculadas al usuario. El pedido guarda una copia (snapshot).
service_tiers	Servicios	Precio en céntimos, plazo, límite declarado y versionado.
card_identities	Carta canónica	Juego, set, número e idioma. Base para la población y las estadísticas.
orders	Pedido comercial	Estado, método de envío, totales congelados y estado de pago.
order_items	Cartas del pedido	Precio y límite congelados al crear el pedido.
item_images	Fotos de cada carta	Clave de almacenamiento, hash SHA-256 y tipo (frente, dorso, extra).
payments	Pagos	Referencia de Stripe, importe y estado.
webhook_events	Eventos de Stripe	El event_id es único, lo que garantiza idempotencia.
shipments	Envíos de entrada y salida	Transportista, número de seguimiento, etiqueta y valor asegurado.
custody_events	Historial de custodia	Solo inserción. El seguimiento público se deriva de aquí.

grading_results	Resultado de grading	Versionado: nunca se sobrescribe, la versión anterior se marca como sustituida.
grading_defects	Defectos	Lado, categoría, severidad, coordenadas x/y y deducción.
slabs	Encapsulado	Serial GG-xxxxxx, UID de NFC y código QR.
certificates	Certificados	Número GGI-xxxxxx, firma Ed25519 y estado (activo, revocado, sustituido). Nunca se borran.
certificate_checks	Verificaciones públicas	Canal (web, QR o NFC) e IP hasheada, para detectar abuso.
events	Eventos y puntos de entrega	Ciudad, recinto, fechas y tipo (feria o hub).
contact_messages	Contacto	Ticket GG-TKT-xxxxxx.
newsletter_subscriptions	Newsletter	Doble confirmación (double opt-in) y consentimiento con fecha y versión.
consents	Consentimientos	Versión de términos, cookies y privacidad aceptada.
email_outbox	Emails pendientes	Plantilla, número de intentos y estado.
audit_logs	Auditoría	Quién cambió qué y cuándo, sobre todo en el back-office.

Reglas del modelo: los precios se guardan en céntimos enteros; la población se calcula por consulta y no se almacena; los certificados son inmutables (se revocan, no se borran).

Estados del pedido: los 8 actuales (SUBMITTED, PACKAGE_RECEIVED, PRE_INSPECTION, OPTICAL_GRADING, ENCAPSULATION, QUALITY_ASSURANCE, SHIPPED, DELIVERED) más los que faltan: PAYMENT_PENDING, ON_HOLD (incidencia), CANCELLED y RETURNED.

5. Reglas de negocio que hay que cerrar con el laboratorio

1. **Fuente única de precios** por servicio, con fecha de vigencia.
2. **Fórmula del seguro** y si el IVA va incluido o se añade.
3. **Límite declarado por servicio:** qué ocurre si una carta lo supera (rechazo o aviso de cambio de nivel, como indica la FAQ).
4. **Derivación de la nota** a partir de los subgrados, y criterio de "Pristine 10" frente a "Gem Mint 10".
5. **Plazos** en días hábiles, con festivos de España, Portugal y Francia.
6. **Transiciones de estado permitidas** y quién puede ejecutarlas.
7. **Cancelación y reembolso**, de acuerdo con la página legal.
8. **Servicio Event:** solo entrega presencial en eventos oficiales.
9. **Formatos de identificador:** GG-ORD, GG-SUB, GG-TKT, GGI (certificado) y serial del slab.

6. API v1 (contratos a implementar)

```
Autenticación
  POST   /api/v1/auth/otp               pedir código por email
  POST   /api/v1/auth/otp/verify        validar código y crear sesión
  POST   /api/v1/auth/logout
  GET    /api/v1/me

Catálogo
  GET    /api/v1/services               servicios activos
  POST   /api/v1/quotes                 cotización calculada en servidor

Pedidos (sesión requerida)
  POST   /api/v1/orders                 crear borrador
  PUT    /api/v1/orders/:id/items       sustituir las cartas del borrador
  POST   /api/v1/orders/:id/items/:itemId/images   URL firmada de subida
  POST   /api/v1/orders/:id/submit      validar y crear sesión de pago
  GET    /api/v1/orders                 pedidos del usuario
  GET    /api/v1/orders/:id             detalle del usuario

Público
  GET    /api/v1/tracking/:code         estado e historial, sin datos personales
  GET    /api/v1/certificates/:num      verificación (404 si no existe)
  GET    /api/v1/certificates/:num/report   informe y defectos
  POST   /api/v1/contact
  POST   /api/v1/newsletter
  GET    /api/v1/events

Webhooks
  POST   /api/v1/webhooks/stripe        firma verificada, idempotente

Back-office (rol requerido)
  GET    /api/v1/admin/orders
  POST   /api/v1/admin/orders/:id/custody-events
  POST   /api/v1/admin/orders/:id/grading-results
  POST   /api/v1/admin/grading-results/:id/approve     doble firma de QA
  POST   /api/v1/admin/certificates                    emitir (firmar)
  POST   /api/v1/admin/certificates/:num/revoke
```

Los errores usan el formato `application/problem+json`. Todos los importes se envían en céntimos.

7. Flujo de extremo a extremo

```
SUBMITTED → PAYMENT_PENDING → (pago correcto) → PACKAGE_RECEIVED
          → PRE_INSPECTION → OPTICAL_GRADING → ENCAPSULATION
          → QUALITY_ASSURANCE (doble firma) → SHIPPED → DELIVERED

Cualquier estado activo → ON_HOLD (incidencia) → estado anterior o CANCELLED
```

Cada transición crea un registro en `custody_events` y un email en la outbox. El seguimiento público muestra solo el historial, nunca notas internas ni la dirección.

8. Seguridad y cumplimiento

- **OTP:** códigos hasheados, caducidad de 10 minutos, máximo 5 intentos y límite de peticiones por email e IP.
- **Autorización:** control de acceso por rol en el servidor. Nunca confiar en el cliente.
- **Webhooks:** verificación de firma e idempotencia por event_id.
- **Fotos:** privadas, con URLs firmadas de vida corta.
- **Certificados:** firma Ed25519 sobre el registro canónico. La clave pública se publica para que cualquiera pueda verificarla.
- **RGPD:** minimización de datos, consentimiento con versión, derecho de acceso y supresión, y contrato de encargado de tratamiento (DPA) con cada proveedor. Hay que definir la retención de fotos y datos de envío.
- **Secretos:** solo en variables de entorno de Vercel. Hoy el proyecto no tiene `.env.example`.
- **Auditoría:** toda acción del back-office queda en `audit_logs`.
- **Copias de seguridad:** recuperación a un punto en el tiempo y prueba de restauración periódica.
- **Seguros y responsabilidad:** decisión abierta del negocio, recogida en el decision log.

9. Integración con el frontend

1. Sustituir `src/data/*` por un cliente de API tipado en `src/api/`, con TanStack Query.
2. Reemplazar `AuthContext` por sesión en cookie y eliminar `gorilla_auth_email` de localStorage.
3. Conectar los campos del paso 3 y trasladar el cálculo de precio al servidor. El wizard solo muestra el resultado de `/quotes`.
4. La ruta `/certificates/:num` debe mostrar 404 para IDs inexistentes.
5. Unificar los precios: una única fuente, `/services`.
6. Retirar TeamVisualMapPage de producción o protegerlo tras autenticación de staff.
7. Ajustar `vercel.json` para excluir `/api` de la reescritura.

10. Calidad, pruebas y CI

- **Unitarias:** motor de precios, seguro, derivación de nota, máquina de estados y validación de límites.
- **Integración:** API contra PostgreSQL real (Docker o testcontainers).
- **Contrato:** especificación OpenAPI generada desde los schemas Zod.
- **E2E (Playwright):** enviar pedido, seguir pedido y verificar certificado.
- **Webhooks:** pruebas de reenvío con Stripe CLI.
- **Seguridad:** pruebas de autorización por rol (IDOR en pedidos y seguimiento) y de límites del OTP.
- **CI:** `tsc`, pruebas y build en cada PR, con previews de Vercel por rama.
- **Carga:** endpoint de verificación pública, que es el más expuesto.

11. Fases y entregables

Estimación orientativa para una persona a tiempo completo. Depende de las decisiones de la sección 12.

Fase	Entregable	Criterio de aceptación	Semanas
0. Base	Repositorio con `api/`, esquema, migraciones, seeds desde los mocks y CI	Migración limpia sobre base vacía; CI en verde	1
1. Cuentas	OTP, sesión y `/me`	El código caduca, se bloquea tras 5 intentos y la cookie es httpOnly	1–2
2. Catálogo	`/services` y `/quotes`	Pruebas de precio y seguro; el frontend no hace cálculos propios	1
3. Pedidos	Borrador, cartas, fotos y envío	Límite por servicio aplicado; IDOR probado	2
4. Pagos	Stripe Checkout y webhook	Un pago duplicado no genera dos pedidos	1–2
5. Notificaciones	Outbox y plantillas	Cada email se envía una sola vez, con reintentos	1
6. Back-office	Custodia, grading y doble firma	Transiciones inválidas rechazadas; auditoría completa	2–3
7. Certificados	Emisión firmada, verificación, QR y NFC	Firma verificable; 404 real; sin datos personales	2
8. Web pública	Contacto, newsletter y eventos	Doble confirmación en newsletter; mensajes guardados	1
9. Salida	Hardening, RGPD, pruebas de carga y de restauración	Checklist de seguridad aprobado	2

Total orientativo: 14 a 18 semanas.

12. Decisiones pendientes

1. **Stack.** Confirmar TypeScript, Hono y PostgreSQL en Vercel. Recomendación: sí.
2. **Momento del pago.** ¿Se cobra al enviar el pedido o al devolverlo? Cambia el flujo y la máquina de estados.
3. **Cuenta.** ¿Se permite enviar sin cuenta? Recomendación: exigir cuenta para ver pedidos, pero permitir pagar como invitado con email verificado.
4. **Precios.** ¿IVA incluido o añadido? ¿El seguro va dentro del precio?
5. **Regla de nota.** Criterio del laboratorio para derivar la nota y para "Pristine 10".
6. **Transportistas.** ¿Integración con GLS y CTT en la primera fase o etiquetas manuales?
7. **Mapa de equipo.** ¿Se elimina o se mueve a un panel interno?

> **Próximo paso.** Con las decisiones de la sección 12 cerradas, el trabajo empieza por la Fase 0.